

Architecture Audit

SEO Analyser — Current Implementation Snapshot

Generated: 2026-07-20 · Documentation only

Architecture Audit — SEO Analyser

Snapshot of the implementation as it exists today. Documentation only; no code was changed.

1. Executive Summary

SEO Analyser is a Node/Express + Postgres backend and Vite/React frontend for Apple Search Ads-style campaign analytics.

CSV uploads land in imports / import_rows, then are rolled into daily_campaign_metrics and daily_keyword_metrics. Most Dashboard, Campaigns, and Keywords period views read those daily tables through analyticsService.js (via compareStructured.js). Campaign segments live in a campaigns dimension table, auto-classified on first insert and user-editable via PATCH /api/campaigns/:id. Keywords inherit segment at query time via a name+app join.

The system still carries dual analytics paths: period-compare over daily tables (primary) and per-import summaries over raw rows (History / Keywords fallback). Brand vs Non-brand on the Dashboard is aggregated in the browser. campaign_id on keyword metrics is unused (always null); relationships use (app_id, campaign_name).

2. Current Architecture

Project structure

Area	Role
backend/	Express API, Postgres pool, migrations, analytics, import, insights, goals
frontend/	React SPA (Vite), pages, shared UI, AppContext
Phase / notes markdown	Historical design notes (not runtime)
Backend services (modules)	
Module	Role
index.js	Routes, multer, startup
db.js	Pool, schema, migrations, startup backfill
imports.js	CSV parse, import CRUD, import summaries
dailyMetrics.js	Build/upsert daily campaign & keyword metrics
campaigns.js	Campaign dimension + segment classify/edit
analyticsService.js	Central queries over daily tables
analyticsMetrics.js	Derived metric formulas (+ older unused helpers)

compareStructured.js	GET /api/compare/period facade
compare.js	Import-vs-import compare; legacy period compare still exported
campaignWeekly.js	Daily/weekly chart data
insightsEngine.js	Rule-based insights
goals.js / alerts.js	Target CRUD / live alert evaluation
annotations.js	Notes CRUD
applIdentity.js	App id/name resolution from CSV

Frontend pages

Page	Role
Dashboard	Period KPIs, charts, brand split, insights, alerts
Campaigns	Campaign table + editable segments + notes/goals drawer
Keywords	Keyword/bid table (period or import fallback)
History	CSV upload + inspect selected import

Shared UI (selected)

BrandCards, KpiCard, TrendChart, WeeklyTrendTable, OverallSummary, CollapsibleSection, CampaignPerformanceTable, EditableSegment, NotesPanel, GoalManagementPanel, DataTable, toolbars/headers.

3. Database

There is no daily_app_metrics or standalone keywords table. Keywords

imports

- Purpose: Uploaded CSV metadata
- PK: id
- Columns: original_name, status, row_count, column_headers, created_at
- Relationships: Parent of import_rows

import_rows

- Purpose: Raw row JSONB payloads
- PK: id
- Columns: import_id, row_number, data, record_key, source_import_id
- Relationships: FK to imports; source for daily metric builds; unique (import_id, row_number)

daily_campaign_metrics

- Purpose: Per-day campaign rollups (primary analytics store)
- PK: id; unique (app_id, report_date, campaign_name)

- Columns: app_id, app_name, campaign_name, report_date, spend/impressions/taps/installs, install attribution columns, daily_budget
- Relationships: Soft-linked to campaigns by (app_id, campaign_name) — no campaign_id column

daily_keyword_metrics

- Purpose: Per-day keyword rollups
- PK: id; unique on app/date/campaign/ad_group/keyword/bid strategy
- Columns: same metric family + ad_group_name, keyword_text, keyword_max_cpt_bid, nullable campaign_id / keyword_id
- Relationships: Soft-linked to campaigns by name+app; campaign_id written as null at import

campaigns

- Purpose: Campaign dimension + editable segment
- PK: id; unique (app_id, campaign_name) when constraint applied
- Columns: app_id, campaign_name, campaign_id (varchar, typically null), segment, timestamps
- Relationships: Joined from daily tables by name+app; created by ensureCampaign on campaign upsert

annotations

- Purpose: Notes on campaign/keyword entities
- PK: id
- Columns: entity_type, entity_key, note_type, note_text, is_pinned
- Relationships: Logical via entity_key string (no FK)

performance_goals

- Purpose: Threshold rules (“targets”)
- PK: id
- Columns: entity_type, entity_key, metric, operator, threshold, period_days, is_active
- Relationships: Logical via entity_key; alerts computed at read time (not stored)

Legacy

- keyword_metrics / keyword_metrics_old — migration only (migrateToDailyTables)

4. Import Pipeline

Actual flow today:

```

CSV Upload (POST /api/imports, multer memory)
! "
parseCsv (imports.js)
! "
createImport – BEGIN transaction
! "
INSERT imports + UPSERT import_rows (ON CONFLICT record_key)
! "
upsertDailyMetricsForImport (same transaction)
! "
buildDailyCampaignRecords / buildDailyKeywordRecords (dailyMetrics.js)
! "
ensureCampaign (classify only if new) + upsert daily_campaign_metrics
! "
upsert daily_keyword_metrics (campaign_id: null)
! "
COMMIT

```

Also at startup (after migrations):
 initDb !' setImmediate !' backfillDailyMetricsFromImportRows
 (reprocesses all import_ids into daily tables)

Validation is light: CSV extension/size via multer; column detection by alias lists; empty/missing metrics treated as zeros/nulls rather than hard rejects.

Edited campaign segments are not overwritten on re-import (ensureCampaign returns existing row unchanged).

5. Analytics Service

Location: backend/analyticsService.js

Formulas: backend/analyticsMetrics.js !' calculateDerivedMetrics (CPA, CPT, TTR, CR; installs = tap-through with fallback)

Responsibilities:

- Period resolution (resolvePeriods)
- Dashboard overall summary
- Campaign / keyword summaries (with optional compare)
- Daily trend, app breakdown, apps list, bid-change scan
- JOINS to campaigns for segment on campaign & keyword queries

Consumers:

Consumer	Path
Dashboard period payload	compareStructured.getPeriodCompare !' analytics service
Weekly charts	campaignWeekly !' getDailyTrend
Insights	insightsEngine !' getDashboardSummary
Alerts	alerts !' getCampaignSummary / getKeywordSummary
Campaigns / Keywords (period mode)	same period compare payload via AppContext

Still outside the analytics service:

- Import-level summaries (getImport*Summary over import_rows)
- Import-vs-import compare (compare.js / GET /api/compare)
- Brand/Non-brand aggregation (frontend computeBrandSplit)
- Weekly WoW deltas in WeeklyTrendTable
- Name-based segment classification at campaign create (campaigns.classifyCampaignSegment)
- Unused overlap: analyticsService.getPeriodComparison vs compareStructured.getPeriodCompare; unused helpers in analyticsMetrics.js / legacy compare.getPeriodCompare

6. Dashboard

Widget	API	Backend	Source / calculation
KPI cards (Spend, Installs, CPA, CPT + secondary)	Context !• GET /api/compare/period	compareStructured !' getDashboardSummary	daily_campaign_metrics sums + calculateDerivedMetrics; sparklines from weekly days
Trend charts	GET /api/campaigns/weekly-performance	campaignWeekly !' getDailyTrend	Daily series from daily_campaign_metrics; SVG client-side
Weekly performance table	Same weekly endpoint	Same	Days rolled to Monday weeks in campaignWeekly; WoW deltas client-side
Brand / Non Brand	Client from periodComparison.campaigns	Campaigns already include segment from JOIN	computeBrandSplit: segment === 'Brand' vs rest
Overall summary	GET /api/insights	insightsEngine	Rule classification on overall metric deltas
Detailed insights	Same	Same	Top scored metric insights; cards mapped client-side
Targets / alerts	GET /api/alerts	alerts.js	Live eval of performance_goals vs analytics summaries
Top campaigns / keywords	Period compare payload	Campaign/keyword summaries	Slice + filterByApp client-side
App breakdown	Period payload	getAppBreakdown	daily_campaign_metrics by app
Annotations	Not a dashboard section	—	Via notes on Campaigns/ Keywords tables
Target CRUD UI	Not on dashboard list	—	GoalManagementPanel from campaign detail drawer

7. Campaigns

- Metrics: periodComparison.campaign preset.
- Segments loaded: SQL LEFT JOIN c
- Edits: EditableSegmentPill !' PATCH / for immediate UI (period cache not refe
- Identity: Stable key for join/edit is inter uniqueness remains (app_id, campaign practice.

- Notes / goals: NotesPanel, CampaignDetailDrawer !' GoalManagementPanel.

8. Keywords

- Metrics source (dual): 1. Prefer periodComparison.keywords (daily tables via analytics service) 2. Else keywordSummary from selected import (getImportKeywordSummary, which also looks up campaigns.segment)
- Campaign relationship: JOIN campaigns ON app_id + campaign_name (not campaign_id)
- Segment: Inherited from campaigns.segment at query time; unmatched !' API null !' UI Unclassified. No keyword-stored segment. No frontend name classifier.
- Bid history: Period path exposes current_bid / previous_bid / bid_change from keyword_max_cpt_bid across periods; import path has current bid only when column present
- Filters: Local FilterBar (app by name, campaign, segment values including Unclassified) — independent of global Dashboard appFilter

9. Insights

- File: insightsEngine.js
- Inputs: Period window (+ optional appld); data from getDashboardSummary(..., compare: true)
- Outputs: { overallInsight, insights[], generatedAt }
- Overall summary: Classify stable / improvement / decline / mixed ("H5% thresholds); pick top drivers; build explanation
- Detailed insights: Metric movers above threshold; weighted score; top 5
- Relationship to Analytics Service: Read-only consumer of overall summary metrics; does not query campaigns/keywords tables itself; deterministic rules (no LLM)

10. Targets

- Storage: performance_goals via goals.js CRUD APIs
- Evaluation: alerts.getAlerts — not persisted; fetches active goals, loads entity metrics from analytics summaries by entity_key, evaluates operator/threshold for configured period_days
- UI appearance: Dashboard alerts list (DashboardAlerts); create/edit/disable/delete in GoalManagementPanel (campaign detail drawer)
- Flow: Fetch goals for entity !' create/update/delete !' alerts re-evaluated on next GET /api/alerts

11. API Map (analytics-relevant)

Endpoint	Purpose	Consumes	Returns
GET /api/compare/period	Main period analytics bundle	days or date range, optional appld	periods, overall, campaigns, keywords, apps, top_*
GET /api/campaigns/weekly-performance	Chart / weekly table	dates, optional campaign/app	days, weeks
GET /api/insights	Overall + detail insights	period + optional appld	overallInsight, insights
GET /api/alerts	Breached goals	optional entity filter	alert list

9. Unused / dead surface — PeriodToolbar.jsx unused; campaignSpendChangePercent unused; large parts of analyticsMetrics.js unused by routes; upsertDailyMetricsForImport not exported.
10. Duplicate filter UX — Keywords FilterBar repeats period/compare/app concepts already in global context/toolbars; Keywords app filter by name vs global by app_key.

14. Architectural Risks

Risk	Why it matters
Multiple sources of truth for “current metrics”	Period daily tables vs last selected import summaries can disagree
Segment join key is name+app only	Renames or whitespace drift break inheritance; no FK integrity
Frontend brand split	Any API/shape change or stale period cache skews Brand cards independently of SQL truth
Alerts not persisted	Harder to audit history; depends on live metric/entity_key match
Classification only on first insert	Correct for edits, but mis-named new campaigns get sticky wrong default until manual edit
periodCompareEnabled does not gate fetch	Toggle only affects UI deltas; easy to misread as “compare off = no period data”
Backend process / code drift	Previously observed: stale Node process serving APIs without segment while disk code had the join

15. Improvement Opportunities

High priority

- Single analytics read path for Dashboard / Campaigns / Keywords (those pages)
- Server-side Brand/Non-brand (or segment rollup) endpoint to remove redundancy
- Stable campaign identity: populate and join on a real campaign ID as the permanent key
- Refetch/invalidate period comparison after segment PATCH

Medium priority

- Unify null-segment display (Other vs Unclassified)
- Retire or isolate legacy compare.getPeriodCompare / unused getPeriodCompare
- Reduce startup full backfill (incremental or flag-gated)
- Align Keywords filtering with global appFilter / app_key

Low priority

- Remove dead frontend components/utils and unused analyticsMetrics
- Persist alert evaluation history if product needs it
- Document entity_key format as a contract

(Listed for planning only — not implemented in this audit.)

16. Important Files

```
backend/
  index.js          # routes, multer, listen
  db.js             # schema, migrations, startup backfill
  imports.js        # CSV !' import_rows !' daily upsert trigger
  dailyMetrics.js   # daily table builders/upserts
  campaigns.js      # segment classify + PATCH support
  analyticsService.js # central daily-table analytics
  analyticsMetrics.js # derived metric formulas
  compareStructured.js # /api/compare/period
  compare.js        # import compare + legacy period
  campaignWeekly.js # weekly/daily chart API
  insightsEngine.js # overall + detailed insights
  goals.js / alerts.js # targets + live alerts
  annotations.js    # notes
  appIdentity.js    # app resolution from CSV

frontend/src/
  context/AppContext.jsx # global period + import state
  api.js                 # HTTP client
  pages/Dashboard.jsx
  pages/Campaigns.jsx
  pages/Keywords.jsx
  pages/History.jsx
  components/BrandCards.jsx
  components/OverallSummary.jsx
  components/InsightsBar.jsx
  components/DashboardAlerts.jsx
  components/EditableSegmentPill.jsx
  components/CampaignPerformanceTable.jsx
  components/KeywordBidTable.jsx
  components/GoalManagementPanel.jsx
  components/NotesPanel.jsx
  utils/dashboardHelpers.js # computeBrandSplit, insight card map
  utils/campaignAnalysis.js
  utils/keywordAnalysis.js
  utils/appFilter.js
```

End of audit. This reflects the running design as implemented: daily-table analytics with a campaigns dimension for segments, a still-active import-summary side path, and several client-side aggregations that sit outside the analytics service.